

EXPERIMENT 1

8 Puzzle Problem

Aim

To write a Python program to solve the **8 Puzzle Problem** using the **Breadth-First Search (BFS)** algorithm

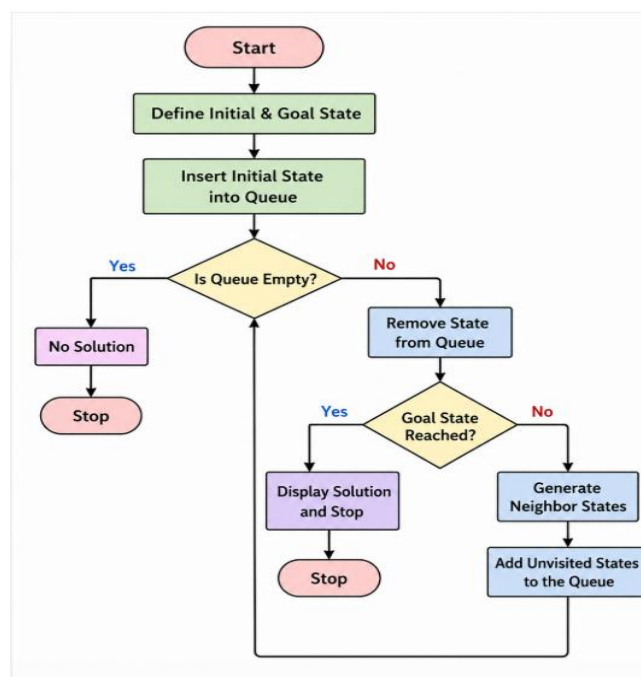
Objective

The objective of this experiment is to solve the 8 Puzzle Problem by finding the shortest sequence of moves that transforms the initial state into the goal state using the Breadth-First Search (BFS) algorithm

Algorithm

1. Define the initial state and the goal state of the puzzle.
2. Place the initial state into a queue.
3. Mark the initial state as visited.
4. Remove the first state from the queue.
5. Check whether it matches the goal state.
6. If it is the goal state, display the solution path and stop.
7. Otherwise, generate all valid neighboring states by moving the blank tile (0) up, down, left, or right.
8. Add all unvisited neighboring states to the queue.
9. Repeat Steps 4–8 until the goal state is found or the queue becomes empty.
10. Display the sequence of moves from the initial state to the goal state.

Flowchart



Python code:

```
from collections import deque
```

```
goal_state = [[1, 2, 3],  
              [4, 5, 6],  
              [7, 8, 0]]
```

```
def find_zero(state):  
    for i in range(3):  
        for j in range(3):  
            if state[i][j] == 0:  
                return i, j
```

```
def get_neighbors(state):  
    neighbors = []  
    x, y = find_zero(state)  
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]  
    for dx, dy in moves:  
        nx, ny = x + dx, y + dy  
        if 0 <= nx < 3 and 0 <= ny < 3:  
            new_state = [row[:] for row in state]  
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]  
            neighbors.append(new_state)  
    return neighbors
```

```
def bfs(start_state):  
    visited = set()  
    queue = deque([(start_state, [])])  
    while queue:  
        state, path = queue.popleft()
```

```
state_tuple = tuple(map(tuple, state))
if state == goal_state:
    return path + [state]
if state_tuple in visited:
    continue
visited.add(state_tuple)
for neighbor in get_neighbors(state):
    queue.append((neighbor, path + [state]))
return None
```

```
def print_state(state):
    for row in state:
        print(row)
    print()
```

```
initial_state = [[1, 2, 3],
                  [4, 0, 6],
                  [7, 5, 8]]
```

```
print("Initial State:")
print_state(initial_state)
```

```
solution = bfs(initial_state)
if solution:
    print("Solution found in", len(solution) - 1, "moves:\n")
    for step in solution:
        print_state(step)
else:
    print("No solution exists.")
```

Output:

Solution found in 2 moves:

[1, 2, 3]

[4, 0, 6]

[7, 5, 8]

[1, 2, 3]

[4, 5, 6]

[7, 0, 8]

[1, 2, 3]

[4, 5, 6]

[7, 8, 0]

Result

The Water Jug Problem was solved successfully using Python.

Conclusion

The program demonstrates state space search to obtain the required quantity of water efficiently.